Cross-Fitted Meta-Learning Classifiers: Probability Stacking and Statistical Meta-Features

Jabulani Monchwe
Applied Business Analytics and Decision Making
North-West University
IDEA-Research Club Chairperson
Johannesburg, South Africa
monchwej.25@gmail.com

Martin Chanza
Applied Business Analytics and Decision Making
North-West University
Mafikeng, South Africa
martin.chanza@nwu.ac.za

Olorato Motitswane
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
oloratomotitswane@gmail.com

Mfundo Monchwe
Eduvos Midrand Campus – IT Faculty Data Science, South Africa
IDEA-Research Club
diditmfundo@gmail.com

***Abstract*—ReliableML implements scikit-learn-compatible meta-learning classifiers for combining base-model evidence. Its general stacking estimators construct class-aligned out-of-fold probabilities, fit a second-stage learner, and retain base estimators refitted on all training rows for inference. Additional estimators select a candidate by stratified cross-validation, calibrate a stacked classifier, or construct LDA-, tree-, and cluster-derived features for a statistical logistic meta-model. This paper formalizes those procedures and distinguishes their implemented scope from broader few-shot or task-adaptation meta-learning. Cross-fitting mitigates a specific source of target leakage but does not eliminate selection bias, distribution shift, or the need for external evaluation.**

***Index Terms*—stacked generalization, cross-fitting, meta-learning, probability calibration, classifier selection**

I. Introduction

Stacked generalization learns a second-level predictor from predictions made by first-level models [1], [2]. Its validity depends on how the second-level training data are constructed: a base learner's in-sample fitted probabilities can encode the target labels too strongly and misrepresent the outputs it will produce on new data. ReliableML's wrappers generate out-of-fold (OOF) meta-features to reduce this discrepancy [5]. The contribution described here is an implementation of known ensemble and statistical-learning ideas, not a claim that the algorithms outperform individual estimators.

II. General cross-fitted stacking

Let $D = \{(x_i, y_i)\}_{i=1}^{n}$, classes $\mathcal{C} = \{1, \ldots, C\}$, base estimators $f_1, \ldots, f_M$, and a stratified partition into K folds $I_1, \ldots, I_K$. For each fold k and model m, fit $f_m^{(-k)}$ on $D \setminus I_k$. For each $i \in I_k$, store its aligned class probability vector

$$z_{im} = \left(\hat{p}_{f_m^{(-k)}}(Y = c \mid x_i) \right)_{c \in \mathcal{C}}.$$

Concatenate $z_{i1}, \ldots, z_{iM}$ into z_i, and fit the meta-estimator g on $\{(z_i, y_i)\}$. At prediction time, refit each base learner f_m on all training data, form $z(x) = [\hat{p}_{f_1}(Y \mid x), \ldots, \hat{p}_{f_M}(Y \mid x)]$, and return $g(z(x))$. Class-column alignment is necessary because a fold learner may expose class columns in a different order or encounter an absent class.

`MetaFeatureTransformer` exposes the OOF transformation and corresponding full-data base models. `MetaPipelineClassifier` combines the OOF features with a user-selected second-stage estimator (default logistic regression). `CalibratedStackedClassifier` fits the stack and then cross-validates sigmoid or isotonic calibration for its meta-estimator. Calibration is learned on the meta-feature training data; final calibration quality must still be measured on independent data.

III. Candidate selection

`PerfMetaSelect` evaluates each candidate learner on the same stratified folds using a user-selected scikit-learn scoring rule. For candidate m, the stored score is

$$\bar{s}_m = \frac{1}{K} \sum_{k=1}^{K} s\left(f_m^{(-k)}, D_k \right), \qquad m^\star = \arg\max_m \bar{s}_m.$$

The selected learner is then cloned and refit on all supplied training rows. This is cross-validated model selection, not

stacking: no second-stage learner is fitted. The estimated maximum is subject to selection optimism, particularly when many candidates or metrics are explored. An outer validation design is needed for unbiased reporting of the selection procedure.

IV. Statistical meta-feature classifier

The statistical variants map the numeric input to a compact feature vector before logistic estimation. For training values X, the procedure fits a standard scaler and LDA projection. LDA class probabilities are generated out-of-fold; the OOF positive-class score is included as a meta-feature. Optional OOF decision-tree probabilities add another supervised signal. MiniBatchKMeans is fitted on scaled inputs, and distances $d_k(x)$ to its centroids are appended. In nonlinear mode the cluster basis uses

$$\psi_k(x) = \exp(-\gamma_k d_k(x)^2);$$

otherwise raw distances are used. Optional degree-two interaction features are added in nonlinear mode. The resulting meta-vector $u(x)$ enters a binary logistic model:

$$P(Y = 1 \mid x) = \sigma(\beta_0 + \beta^\top u(x)), \qquad \sigma(a) = \frac{1}{1 + \exp(-a)}.$$

The implementation's logistic maximum-likelihood objective, up to a small numerical ridge term, is

$$\ell(\beta) = -\sum_i \{y_i \log p_i + (1 - y_i) \log(1 - p_i)\} + \lambda \|\beta_{-0}\|_2^2.$$

The IRLS option uses iteratively reweighted least squares with working weights $w_i = p_i(1 - p_i)$, score $U = X^\top(y - p)$, and update $\beta^{(t+1)} = \beta^{(t)} + (X^\top W X + \lambda I)^{-1} U$. Alternative estimation options include numerical MLE and elastic-net logistic regression. An optional class-stratified Monte Carlo resampling mode averages coefficients across resampled fits; this adds computation and does not itself provide a calibrated uncertainty interval.

These statistical meta-classifiers currently require binary classification and finite numeric features. The general stacking wrappers are distinct and can support multiclass tasks when the base and meta estimators meet the probability-output requirements. The statistical meta-feature construction uses full-training fitted LDA/cluster transforms for inference, with cross-fitting specifically applied to selected supervised probability meta-features; users should not interpret every feature component as fully cross-fitted.

A. Neural meta-network

`StatisticalMetaNNClassifier` uses the LDA projection, OOF LDA probabilities, optional OOF tree probabilities, and adaptive radial cluster features as inputs to a feed-forward neural meta-model. Its hidden activation is the sigmoid-weighted linear unit $a(z) = z\sigma(z)$. During training, inverted dropout multiplies activations by m, with $m \sim \text{Bernoulli}(1 - r)/(1-r)$ for dropout rate r; the output layer uses a sigmoid to estimate the positive-class probability. Mini-batch optimization uses Adam-style first and second moments with weight decay. With Monte Carlo dropout enabled at inference, the reported probability averages stochastic forward passes,

$$\hat{p}(Y = 1 \mid x) = \frac{1}{B} \sum_{b=1}^{B} \sigma(f_{\theta, m_b}(x)) .$$

This average is a stochastic prediction summary, not a calibrated epistemic-uncertainty interval. The neural estimator is implemented in NumPy, supports binary classification only, and shares the finite-numeric-input and validation cautions of the other statistical variants.

V. Search, leakage control, and evaluation

Successive-halving grid and randomized-search helpers are provided through scikit-learn [4]. Data-dependent preprocessing should be nested inside each estimator so it is refit within each training fold. Resampling must likewise be confined to training folds. Select the primary score before inspecting results; choose metrics suited to class prevalence and deployment cost, and separately evaluate probability calibration when probabilities affect decisions [3].

Cross-fitting prevents each training row's supervised OOF meta-feature from being generated by a base model trained on that row. It does not protect against leakage in upstream preprocessing, feature selection, repeated validation tuning, or a nonrepresentative split. A defensible comparison uses nested cross-validation or an untouched outer test set, matched baseline models, appropriate temporal/group separation, repeated seeds where justified, and paired uncertainty estimates at the independent unit.

VI. Limitations and interpretation

The package provides estimator code, not a benchmark proving superiority of meta-learning. Stacking can add variance, computational cost, and calibration error; correlated base predictions may add little independent information. OOF probabilities vary with folds, and class alignment does not guarantee probability calibration. Binary-only statistical variants, finite numeric input requirements, base learner computation, and small minority classes constrain applicability. The term "meta-learning" here refers principally to learned combination/selection of models, not gradient-based adaptation across task distributions.

VII. Conclusion

ReliableML implements OOF probability stacking, cross-validated candidate selection, calibrated stacking, and binary statistical meta-feature learners. The equations expose how each differs and what its validation must establish. Claims about predictive advantage require empirical comparisons on untouched target-relevant data.

References

[1] D. H. Wolpert, "Stacked generalization," *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.

[2] L. Breiman, "Stacked regressions," *Machine Learning*, vol. 24, pp. 49–64, 1996.

[3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[5] J. Monchwe, O. Motitswane, and M. Monchwe, *ReliableML*, software, version 0.1.1, 2026. https://github.com/Jabualani/ML-Reliabilty.